

MediConnect

Sistema Inteligente Móvil de Gestión de Citas y Recetas
Médicas

Informe Final del Curso de Tecnologías Móviles

Autor:
m4fb

Profesor:
Docente del Curso

30 de junio de 2026

Índice general

1. Planteamiento del Problema y Objetivos	2
1.1. Justificación del Desarrollo de la Aplicación Móvil	2
1.2. Objetivos	3
1.2.1. Objetivo General	3
1.2.2. Objetivos Específicos	3
2. Estado del Arte y Marco Teórico	4
2.1. Estado del Arte	4
2.2. Marco Teórico de Tecnologías Móviles	4
2.2.1. Android Nativo y Kotlin	4
2.2.2. Capa de Persistencia Local: Room Database	4
2.2.3. Inyección de Dependencias: Dagger Hilt	5
2.2.4. Consumo de Servicios RESTful: Retrofit 2 & OkHttp 4	5
2.2.5. Seguridad en el Dispositivo: Biometría y Cifrado	5
2.3. Marco Teórico Backend e Infraestructura	5
2.4. Metodología de Desarrollo: SCRUM	6
3. Análisis de Requerimientos	7
3.1. Requerimientos Funcionales (RF)	7
3.2. Requerimientos No Funcionales (RNF)	9

Capítulo 1

Planteamiento del Problema y Objetivos

1.1. Justificación del Desarrollo de la Aplicación Móvil

El sector de la salud a nivel global experimenta una necesidad urgente de optimización en sus canales de interacción con los pacientes. Los sistemas tradicionales de gestión médica suelen depender de flujos de trabajo centralizados, presenciales o telefónicos, lo que genera tiempos de espera excesivos, cuellos de botella administrativos y una experiencia de usuario deficiente.

La introducción de la aplicación móvil nativa **MediConnect** se justifica directamente bajo los siguientes pilares de impacto sectorial y de mercado:

- **Eficiencia Operativa y Administrativa:** Permite automatizar el flujo de admisión (check-in) del paciente mediante tecnologías de lectura rápida como códigos QR, reduciendo la interacción manual en la recepción y eliminando las colas físicas en salas de espera.
- **Gestión de la Información Médica:** Centraliza de manera segura el acceso a las recetas farmacológicas y el historial de consultas del paciente desde un único dispositivo móvil, evitando la pérdida de recetas físicas y garantizando la trazabilidad de los tratamientos médicos.
- **Soporte Fuera de Línea (Offline Support):** Los pacientes no siempre disponen de una conexión a internet estable cuando se encuentran en tránsito o dentro de infraestructuras hospitalarias complejas. MediConnect almacena localmente los datos más relevantes de la agenda médica y el directorio de doctores, permitiendo al usuario consultar sus citas programadas sin depender de acceso constante a datos móviles.
- **Seguridad de Datos de Salud:** La protección de la privacidad del paciente es crítica. El uso de autenticación biométrica local y almacenamiento cifrado en el dispositivo eleva sustancialmente la barrera de seguridad en comparación con aplicaciones web convencionales.

1.2. Objetivos

1.2.1. Objetivo General

Diseñar, desarrollar y desplegar un sistema integral móvil nativo denominado **MediConnect** que optimice el agendamiento de citas, la consulta de recetas médicas y la seguridad de los flujos de check-in en centros de atención médica mediante una arquitectura móvil robusta y un backend distribuido y escalable.

1.2.2. Objetivos Específicos

1. Desarrollar una aplicación móvil nativa para la plataforma Android usando el lenguaje Kotlin, adoptando el patrón de diseño arquitectónico Model-View-ViewModel (MVVM) y principios de Clean Architecture.
2. Implementar una capa de almacenamiento local estructurado en Android mediante la librería Room (SQLite), permitiendo un soporte fuera de línea (offline cache) reactivo e inteligente para citas y perfiles médicos.
3. Integrar mecanismos de seguridad avanzados a nivel de dispositivo móvil, incluyendo cifrado de tokens de sesión con **EncryptedSharedPreferences** y validación de identidad local vía APIs biométricas (**BiometricPrompt**).
4. Implementar la generación y renderizado dinámico de códigos QR basados en la biblioteca ZXing para agilizar y asegurar el check-in presencial en las citas de los pacientes.
5. Construir un backend robusto basado en microservicios en Java 17 y Spring Boot 3.3.6, documentado con OpenAPI/Swagger y desplegado sobre infraestructura de contenedores orquestada por Kubernetes (K3s).

Capítulo 2

Estado del Arte y Marco Teórico

2.1. Estado del Arte

En el mercado actual existen diversas soluciones dirigidas al sector de salud digital (eHealth), tales como *Doctoralia*, *MyChart* o *Salud.uy*. Si bien estas plataformas facilitan la comunicación y la reserva de citas, presentan limitaciones estructurales en varios aspectos clave:

1. **Falta de Soporte Offline Integral:** La mayoría de estas aplicaciones funcionan como meros visores web (WebViews) encapsulados, lo que causa fallos completos o pantallas en blanco cuando el paciente pierde conexión a internet.
2. **Procesos de Admisión Desarticulados:** Siguen requiriendo el registro manual en ventanilla física al llegar al consultorio, no aprovechando tecnologías rápidas como códigos QR dinámicos para el check-in automático.
3. **Consumo Ineficiente de Recursos:** No estructuran el flujo de datos bajo inyección de dependencias rígida ni modelos reactivos, lo que ralentiza la respuesta en teléfonos de gama media y baja.

2.2. Marco Teórico de Tecnologías Móviles

2.2.1. Android Nativo y Kotlin

El desarrollo nativo asegura el máximo rendimiento y el acceso inmediato a las APIs de hardware de los dispositivos Android. **Kotlin** es el lenguaje de programación recomendado oficialmente por Google debido a su diseño moderno, seguridad contra referencias nulas (null safety) y soporte nativo para programación asíncrona mediante **Corrutinas**, reduciendo la complejidad del código y mejorando la fluidez de la interfaz.

2.2.2. Capa de Persistencia Local: Room Database

Room es una biblioteca de mapeo objeto-relacional (ORM) que proporciona una capa de abstracción sobre SQLite. Permite la validación de consultas SQL en

tiempo de compilación y ofrece retornos reactivos mediante flujos de datos asíncronos (**Kotlin Flow**), siendo clave para mantener sincronizada la interfaz de usuario con los datos almacenados localmente de forma asíncrona.

2.2.3. Inyección de Dependencias: Dagger Hilt

Hilt simplifica la inyección de dependencias en Android al automatizar la creación y el ciclo de vida de los componentes requeridos por las distintas clases (View-Models, Repositorios, APIs). Esto garantiza el bajo acoplamiento del software, facilita la mantenibilidad del código y permite realizar pruebas unitarias más eficaces.

2.2.4. Consumo de Servicios RESTful: Retrofit 2 & OkHttp 4

Retrofit convierte de forma eficiente las interfaces de red de Kotlin en clientes HTTP capaces de interactuar de forma sencilla con el API de Spring Boot. Por su parte, **OkHttp** actúa como el motor cliente subyacente, permitiendo la inyección de interceptores de red para adjuntar cabeceras de autorización JWT en cada petición y manejar de manera centralizada la expiración de la sesión (errores HTTP 401).

2.2.5. Seguridad en el Dispositivo: Biometría y Cifrado

Para la protección de datos médicos locales se emplean dos tecnologías fundamentales:

- **EncryptedSharedPreferences**: Parte de Android Jetpack Security. Utiliza el Android Keystore System para cifrar automáticamente tanto las llaves como los valores de sesión (tokens JWT) con algoritmos AES de 256 bits.
- **BiometricPrompt**: Interfaz unificada provista por el sistema para validar la identidad del usuario a través de lectores de huella dactilar o reconocimiento facial integrados en el hardware.

2.3. Marco Teórico Backend e Infraestructura

El servidor central de MediConnect se construyó bajo la arquitectura de micro-servicios usando **Spring Boot 3**, gestionando la persistencia de datos mediante **Spring Data JPA** hacia una base de datos relacional **PostgreSQL**.

La infraestructura se diseñó para alta disponibilidad empleando **Docker** para la creación de imágenes ligeras del backend y la base de datos, y **Kubernetes (K3s)** para la orquestación y autorrecuperación de los pods de ejecución en producción. La comunicación segura desde el cliente se gestiona mediante un canal privado cifrado a través del túnel VPN de **WireGuard**.

2.4. Metodología de Desarrollo: SCRUM

Para la planificación y ejecución del proyecto se adoptó la metodología ágil **SCRUM**, organizada en iteraciones cortas llamadas **Sprints** de 2 semanas de duración. Las fases y dinámicas integradas en el flujo del proyecto incluyeron:

- **Planificación del Sprint (Sprint Planning):** Selección de tareas del Product Backlog prioritarias para incorporarlas al Sprint Backlog.
- **Reuniones Diarias (Daily Standups):** Breves actualizaciones del equipo de desarrollo para identificar impedimentos.
- **Revisión del Sprint (Sprint Review):** Demostración del incremento funcional desarrollado.
- **Retrospectiva:** Análisis del proceso de trabajo para implementar mejoras operativas.

Capítulo 3

Análisis de Requerimientos

3.1. Requerimientos Funcionales (RF)

Los requerimientos funcionales definen los servicios que el sistema debe proporcionar y cómo debe reaccionar ante entradas específicas.

Cuadro 3.1: Especificación de Requerimientos Funcionales

Código	Descripción del Requerimiento
RF-01	El sistema debe permitir el registro de nuevos pacientes solicitando nombre, apellido, email, contraseña, fecha de nacimiento, género, grupo sanguíneo y dirección.
RF-02	El sistema debe permitir el inicio de sesión y autenticación segura mediante email y contraseña, soportando además validación biométrica en dispositivos compatibles.
RF-03	El sistema debe permitir a los pacientes visualizar una lista de médicos especialistas y filtrar la búsqueda por nombre y/o especialidad.
RF-04	El sistema debe permitir al paciente agendar una cita médica seleccionando un doctor, una fecha, una hora disponible y detallando el motivo de la cita.
RF-05	El sistema debe permitir la cancelación de citas agendadas por parte del paciente, exigiendo ingresar obligatoriamente un motivo de cancelación.
RF-06	El sistema debe permitir generar dinámicamente un código QR para que el paciente realice el check-in (admisión automática) de su cita médica en el consultorio.
RF-07	El sistema debe permitir al paciente visualizar sus recetas médicas activas y el detalle de los medicamentos prescritos.
RF-08	El sistema debe permitir al paciente acceder a su historial clínico digitalizado y agregar de manera manual nuevos registros médicos históricos.
RF-09	El sistema debe permitir mostrar notificaciones de actualizaciones relacionadas al estado de las citas programadas.
RF-10	El sistema debe soportar almacenamiento en caché offline de las citas y la información de los médicos guardados previamente en el dispositivo.
RF-11	El sistema debe permitir al paciente registrar valoraciones y opiniones (de 1 a 5 estrellas) para un doctor con el que haya tenido contacto.
RF-12	El sistema debe permitir al paciente visualizar las valoraciones promedio y comentarios previos de otros usuarios en el perfil detallado del doctor.
RF-13	El sistema debe permitir al paciente consultar dinámicamente los horarios disponibles de un doctor seleccionando un día en particular del calendario.
RF-14	El sistema debe permitir listar todas las notificaciones recibidas en el panel central y marcarlas de forma individual como leídas.
RF-15	El sistema debe permitir al usuario modificar su contraseña de ingreso desde su panel de perfil personal mediante validación previa de seguridad.

3.2. Requerimientos No Funcionales (RNF)

Los requerimientos no funcionales definen propiedades de calidad y restricciones sobre el diseño y la implementación del software.

Cuadro 3.2: Especificación de Requerimientos No Funcionales

Código	Propiedad / Restricción a Cumplir
RNF-01	Seguridad de Datos: Todas las llamadas de red deben transmitirse encriptadas. Las contraseñas en la base de datos deben almacenarse hasheadas con BCrypt.
RNF-02	Seguridad en Dispositivo: El token de sesión JWT debe almacenarse exclusivamente en almacenamiento local cifrado (<code>EncryptedSharedPreferences</code>).
RNF-03	Compatibilidad: La aplicación móvil Android debe ser compatible a partir de la versión de Android 7.0 (API nivel 24) en adelante.
RNF-04	Disponibilidad: El backend debe estar alojado en Kubernetes con políticas de replicación y volumen persistente independiente para asegurar alta disponibilidad ante caídas de nodos.
RNF-05	Rendimiento de Red: Las consultas del cliente móvil al API REST deben completarse en un tiempo promedio menor a 1.5 segundos en condiciones normales de red.
RNF-06	Reactividad UI: El sistema debe mostrar un indicador visual de carga (<code>ProgressBar</code>) mientras se realicen operaciones asíncronas con el backend o la base de datos local.
RNF-07	Eficiencia de Recursos: El consumo de memoria RAM de la aplicación en segundo plano no debe exceder los 50 MB en dispositivos móviles de gama media.
RNF-08	Interoperabilidad: La API RESTful expuesta por el backend debe utilizar formatos estandarizados en JSON para asegurar compatibilidad con futuros clientes móviles (iOS) o web.
RNF-09	Persistencia y Resiliencia: La caché offline local en Room no debe perderse ante el apagado del dispositivo móvil ni ante cierres forzados de la aplicación.